

Sunshine Retail Platform

Project Report

Prepared by: Divya Rachala

Abstract

This report documents an Indian retail project that connects a 50 product Next.js storefront with Java order orchestration, Python recommendations, conversational customer support and reproducible retail analytics. The system covers discovery, cart, checkout, inventory reservation, payment success and failure, fulfilment, an eight stage delivery timeline, profile and recent orders.

The implementation verifies three inventory outcomes: committed after successful payment, released after payment failure and rejected when quantity is unavailable. Twenty three automated tests pass across TypeScript, Java and Python. A declared 12 row synthetic analytics fixture produces a 75.0 percent confirmation rate, INR 37,136 in confirmed revenue and INR 4,126.22 average order value.

Keywords: retail platform; inventory reservation; multi agent orchestration; delivery tracking; Next.js; Java; Python; testing

Report structure

The report presents the problem, project scope, customer journey, system design, inventory and delivery controls, evaluation evidence, limitations, reproducibility details, and conclusion.

1. Problem statement, root cause and purpose

Sunshine is an Indian retail application that covers product discovery, conversational support, cart, checkout, inventory decisions, payment outcomes, fulfilment, delivery tracking, recent orders and retail analytics. I selected this problem because many small commerce projects demonstrate attractive screens but cannot explain the complete order state after a customer selects Place order.

The root cause is disconnected state. Product recommendations may not use the real catalogue, checkout may trust an old stock value, payment failure may leave a unit incorrectly deducted, and order history may contain a delivery promise that was never created. These inconsistencies are important because the customer interface can appear successful even when the underlying rules are incomplete.

The purpose of Sunshine is to connect those decisions through one inspectable project. The customer sees familiar shopping language, while the source code and report preserve the internal agent handoffs, inventory snapshots and failure evidence needed to verify the result.

Project objective	Evidence used
Connect discovery to checkout	One 50 product catalogue supports search, chat, cart and recommendations
Preserve stock correctness	Before, held and after quantity returned for every order line
Explain success and failure	Eight customer milestones and three reproducible checkout branches
Keep the result measurable	23 automated tests and a reproducible 12 row analytics fixture

2. Project scope and data design

The catalogue contains 50 synthetic products across women's fashion, men's fashion, footwear and bags, electronics, and home and living. Each record contains a stable identifier, slug, category, Indian rupee price, list price, rating, colour, delivery time, available quantity and optional sizes. The dataset is intentionally bounded because this project evaluates connected software behaviour rather than a statistical model that benefits from millions of observations.

Public example orders make arriving today, shipped, delivered, payment failed and out of stock states visible on a first visit. Personal orders, cart quantities and stock changes are stored only in the visitor's browser. Versioned storage keys prevent an older response schema from being mixed with the current inventory and milestone contract.

A separate 12 row synthetic raw order file supports the analytics workflow. It contains enough outcome, category, city and revenue variation to demonstrate cleaning, confirmation rate, revenue, average order value and leading segment calculations without implying production scale.

Data area	Scale	Use
Catalogue	50 products, 5 categories	Discovery, chat, cart and stock
Public orders	5 scenario examples	Recent orders and delivery states
Analytics fixture	12 raw order rows	Reproducible KPI calculations
Customer state	Per browser	Cart, personal orders and stock changes

3. Customer journey

A customer can search the catalogue, select a category, review a product, choose a size, add an item to the cart and continue to checkout. The interface uses Indian address fields, INR prices, UPI, card and cash on delivery choices, free delivery rules and an explicit statement that no real payment is collected.

Divya, the customer facing assistant, can interpret a shopping request, ask for missing preferences, recommend verified catalogue products, add a selected size to the cart, explain checkout rules and retrieve an exact order number. Product and order facts are resolved from application data rather than generated by the model.

After checkout, the order page shows a complete journey instead of a single success message. Customers can see which stage is completed, current, upcoming or stopped, together with the inventory result for each ordered product. Recent orders remain available from the profile menu.

4. System architecture and responsibility boundaries

The Next.js 16 and React 19 application owns the storefront, chat interface, cart, checkout, profile, customer milestones and browser persistence. Route handlers provide stable order, recommendation and conversation endpoints. A hosted TypeScript adapter preserves the order contract when the external Java service is not configured on Vercel.

The Java 17 Spring Boot service owns request validation and deterministic order coordination. The Python FastAPI service owns content based recommendation ranking. A pandas script validates the raw analytics fixture and writes governed KPI output. Docker Compose provides a local combined runtime, while GitHub Actions and language specific test runners verify each boundary.

The language model boundary is limited to interpreting conversational intent. Ollama does not create prices, sizes, inventory quantities, order states or delivery dates. If the model is unavailable, built in intent logic provides the same supported website actions.

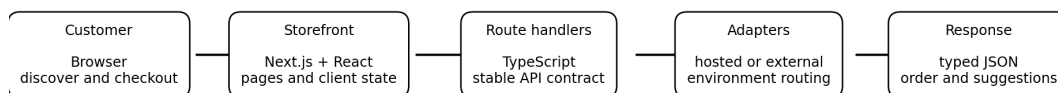
Component	Primary responsibility	Failure boundary
Next.js storefront	Customer interaction and hosted API contract	Displays safe result returned by the service
Spring Boot	Order sequence, inventory outcome and delivery milestones	Stops downstream agents after a required failure
FastAPI	Deterministic related product ranking	Returns validated catalogue suggestions only
pandas workflow	Raw order cleaning and retail KPIs	Rejects invalid analytical inputs
Ollama support	Natural language intent selection	Falls back to bounded local rules

5. Frontend and hosted API architecture

The Next.js application separates server-rendered catalogue pages, client-side cart and profile state, and route handlers. React components render product, checkout, order and support views, while TypeScript types define the payloads exchanged with order, recommendation and conversation endpoints.

Route handlers select an external service when its URL is configured and a contract-compatible hosted adapter otherwise. The customer interface therefore consumes one stable response shape in local Docker development and on the public Vercel deployment.

Frontend and hosted API boundary



One typed application contract supports the browser and external services.

Architecture detail A. Browser, Next.js route handlers, adapters and typed response boundary.

Frontend responsibility: Next.js and React own navigation, interaction and customer explanation, not inventory or payment decisions.

API responsibility: TypeScript route handlers validate and route requests without changing the declared order contract.

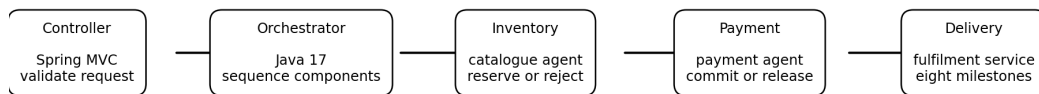
Deployment result: The same UI works with local Java and Python services or the hosted adapters used by Vercel.

6. Java backend order architecture

The Java service begins at a Spring MVC controller and passes a validated request to OrderOrchestrator. Catalogue, risk, payment, fulfilment, delivery and notification components return typed results in dependency order.

A stock rejection stops before risk and payment. A declined payment releases held units and prevents fulfilment and delivery. Only a successful payment commits inventory and creates the eight-stage customer timeline. A fixed clock in the tests keeps delivery dates reproducible.

Java backend order execution



The orchestrator stops later components after a required failure.

Architecture detail B. Spring controller, Java orchestrator and order-state responsibilities.

Backend responsibility: Spring Boot owns validation, orchestration, state transitions and the delivery contract.

Failure control: The orchestrator explicitly marks later components as stopped after rejection or decline.

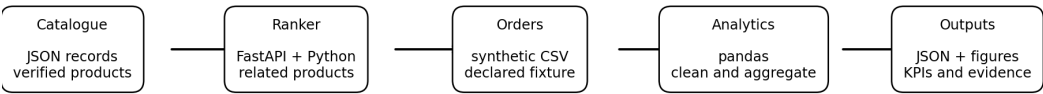
Backend skill: The service demonstrates REST design, Java records, dependency injection, bounded components and JUnit scenario tests.

7. Python recommendation and analytics architecture

The FastAPI service reads the verified catalogue and ranks related products using category, rating and relative price distance. It returns declared product records and never creates a price, stock value or size.

A separate pandas workflow reads the synthetic order fixture, validates required fields, normalises outcomes and writes KPI JSON. Recommendation and analytics services support the customer journey but do not own inventory or order state.

Python recommendation and analytics services



Python supports product ranking and reproducible KPIs without owning order state.

Architecture detail C. Python product-ranking and analytical-output boundaries.

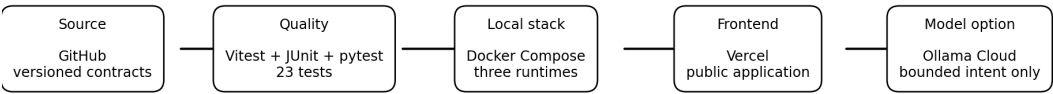
- Recommendation path:** Catalogue JSON passes through FastAPI ranking and returns only verified products.
- Analytics path:** Synthetic order CSV passes through pandas validation before metrics and figures are produced.
- Boundary:** Neither Python path can approve payment, reserve stock or advance a delivery milestone.

8. Deployment and verification architecture

GitHub stores the cross-runtime source and contracts. Vitest, JUnit and pytest verify the responsibilities owned by TypeScript, Java and Python. Docker Compose starts the complete local stack, while Vercel hosts the public Next.js application.

Ollama Cloud is optional and limited to supported intent interpretation. The model key remains on the server. When the model is unavailable, bounded local intent logic keeps catalogue, cart and order support operational.

Build, deployment and service verification



Each runtime can be tested separately or started together for local integration.

Architecture detail D. Source control, quality gates, local integration, public hosting and optional model boundary.

Quality gates: Twenty-three tests run across three runtimes rather than treating a frontend build as complete verification.

Local integration: Docker Compose connects Next.js, Spring Boot and FastAPI with explicit service URLs.

Hosted boundary: Vercel serves the customer application, while optional external services remain independently deployable.

9. Order orchestration experiment

The order service uses six bounded components coordinated by OrderOrchestrator. Catalogue Agent handles the availability reservation, Risk Agent checks deterministic address and order rules, Payment Agent handles the simulated method, Fulfilment Agent plans picking and packing, Delivery Agent creates the tracking contract, and Notification Agent records the final update.

These agents are ordinary Java components rather than language model agents. Each one returns a typed result, which makes the stopping rules visible and independently testable. The customer interface deliberately translates the trace into normal order language and does not expose internal agent names.

Agent outcome by checkout branch						
Confirmed	Completed	Completed	Completed	Completed	Completed	Completed
Payment failed	Completed	Completed	Failed	Skipped	Skipped	Completed
Out of stock	Failed	Skipped	Skipped	Skipped	Skipped	Completed
	Catalogue	Risk	Payment	Fulfilment	Delivery	Notification

Figure 1. Completed, failed and skipped agent outcomes for each checkout branch.

Test question: Does each required failure stop agents that would otherwise create payment, fulfilment or delivery side effects?

Observed result: All six agents complete for confirmation. Payment failure stops fulfilment and delivery. An unavailable quantity stops risk, payment, fulfilment and delivery while notification still records the outcome.

Interpretation: The coordinator produces an explicit partial trace instead of silently continuing after failure. This is the central orchestration evidence in the project.

10. Inventory reservation experiment

A stock number should not be permanently reduced simply because checkout started. Sunshine therefore separates a temporary hold from the final inventory decision. Every response includes a disposition and a per item snapshot containing requested quantity, quantity available before checkout, quantity held and quantity available after the branch completes.

Inventory reservation state transitions

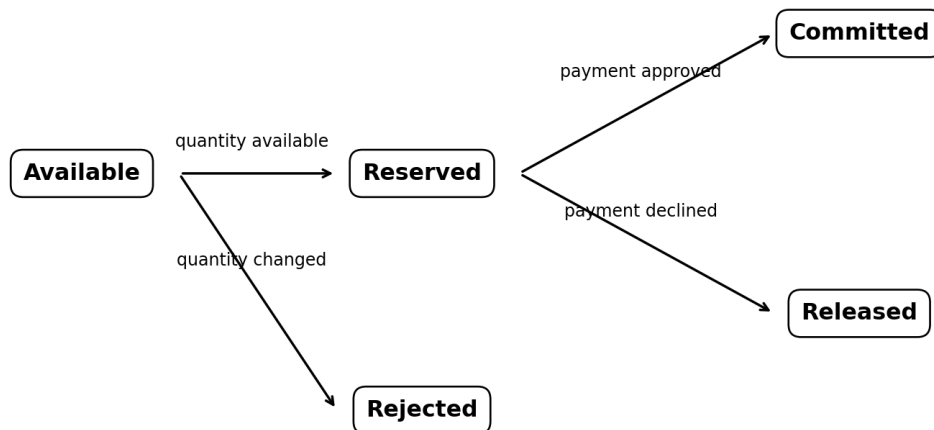


Figure 2. Inventory transitions for confirmed, payment failed and unavailable branches.

Test question: Is stock committed only after payment succeeds, and is the earlier value restored after a decline?

Observed result: Confirmation moves Available to Reserved to Committed. Payment failure moves Available to Reserved to Released. A stock conflict moves Available directly to Rejected before payment.

Interpretation: The state model prevents two common order errors: charging for unavailable products and losing stock after a declined payment.

11. Customer delivery lifecycle experiment

A confirmed order begins a separate customer lifecycle. The same eight stage contract is returned by Java and the hosted adapter. Public examples advance that contract to shipped, out for delivery and delivered so the implementation demonstrates more than the first successful checkout state.

Eight stage customer delivery contract

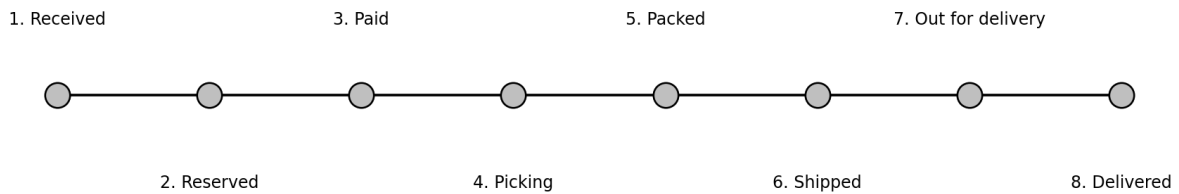


Figure 3. Eight ordered milestones from order receipt through delivery.

Test question: Can every order state be explained using an ordered, typed milestone rather than a loose message?

Observed result: The contract contains order received, items reserved, payment confirmed, picking, packed, shipped, out for delivery and delivered.

Interpretation: A successful new order marks picking as current; failure branches mark the stopping point and every later stage as stopped.

12. Analytics preparation and order outcome experiment

The analytics script reads the raw synthetic order file, validates required columns and normalises confirmed outcomes before calculating summary measures. The raw file is retained separately from the generated KPI output so the calculations can be reproduced and reviewed.

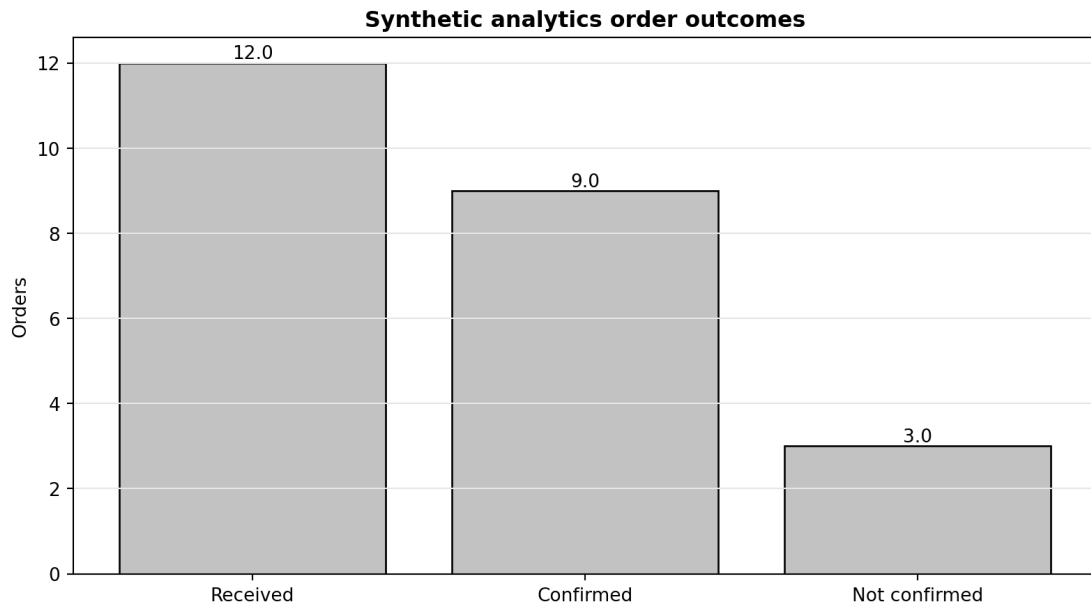


Figure 4. Received, confirmed and non confirmed orders in the analytics fixture.

Test question: Does the KPI workflow reconcile raw status values into a transparent order funnel?

Observed result: Nine of 12 orders are confirmed and three are not confirmed, producing a 75.0 percent confirmation rate.

Interpretation: Failed outcomes remain part of the denominator, preventing the confirmation rate from being overstated.

13. Revenue and average order value experiment

Revenue is calculated only from confirmed synthetic orders. Average order value uses confirmed revenue divided by confirmed order count. This definition avoids treating a failed payment or unavailable order as recognised revenue.

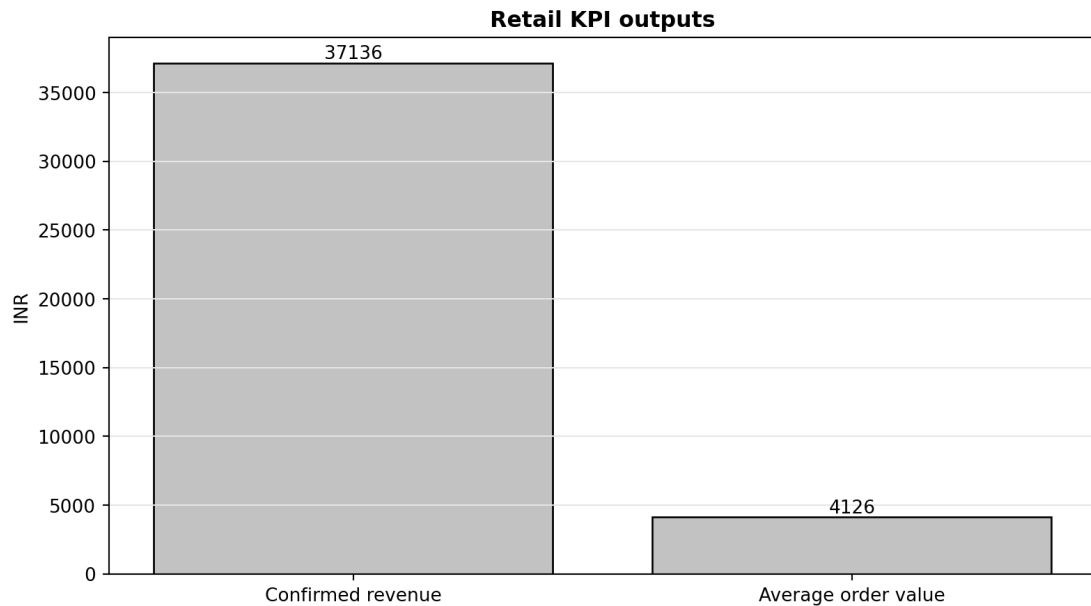


Figure 5. Confirmed revenue and average order value from the reproducible KPI workflow.

Test question: Do the revenue measures respect the same confirmation rule used in the order funnel?

Observed result: Confirmed revenue is INR 37,136 and average order value is INR 4,126.22.

Interpretation: Delhi is the highest revenue city and Electronics is the highest revenue category in this declared synthetic fixture.

14. Automated verification experiment

The test strategy is divided by responsibility. TypeScript tests cover pricing, delivery fees, assistant intent, order lookup, stock outcomes and the hosted contract. JUnit tests cover Java success, rejection, reservation release and milestones. Pytest covers recommendation ranking, validation and API behaviour.

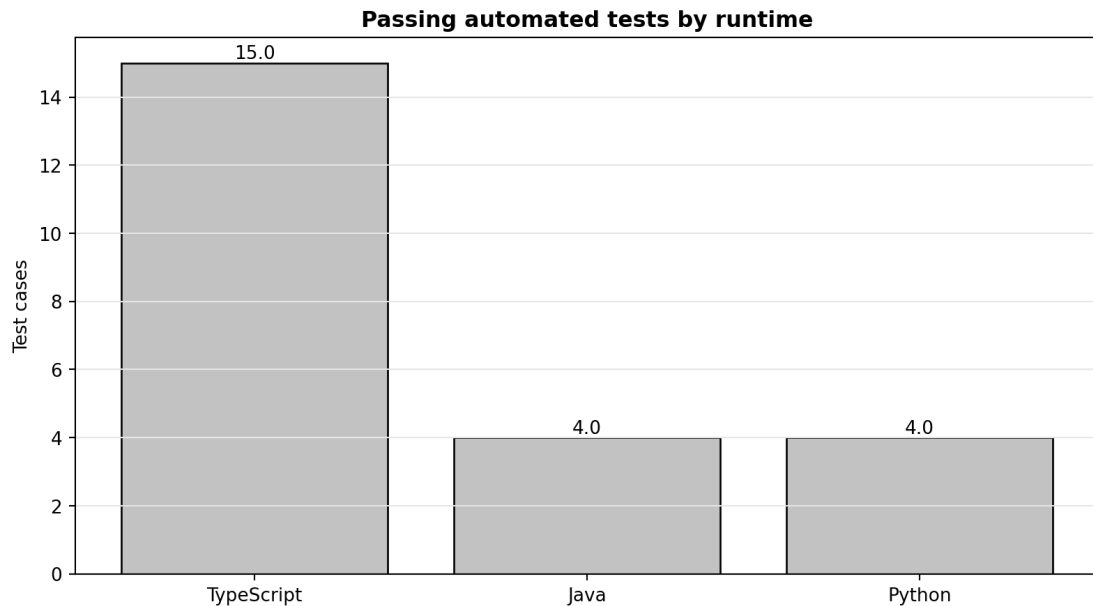


Figure 6. Passing automated tests across TypeScript, Java and Python.

Test question: Are the critical rules checked in the runtime that owns them?

Observed result: Fifteen TypeScript tests, four Java tests and four Python tests pass, giving 23 automated tests in total.

Interpretation: The suite checks both successful results and the side effects that must not occur after failure.

15. Actual cross-runtime test execution

I reran the three current test suites for this report revision. Fifteen TypeScript tests, four JUnit tests and four Python tests passed. No failed test is omitted from the reported total of twenty-three.

The terminal evidence names every runtime because a combined chart cannot show whether one backend was skipped. The scenarios cover success, failure recovery, assistant boundaries, recommendation behaviour and API validation.

```
Actual cross-runtime test execution

$ npm test                15 passed | 0 failed
$ backend-java/mvnw test   4 passed | 0 failed
$ python -m pytest         4 passed | 0 failed

23 automated tests passed across TypeScript, Java and Python.

Verified scenarios: pricing, fees, assistant scope, order lookup,
stock commit, reservation release, rejection, delivery milestones,
recommendation ranking, input validation and API behaviour.
```

Test evidence. Current TypeScript, Java and Python execution results for Sunshine.

TypeScript: Fifteen tests verify price totals, fees, support intent, order lookup, stock results and the hosted contract.

Java: Four JUnit tests verify confirmation, release after decline, rejection before payment and delivery milestones.

Python: Four pytest cases verify ranking, validation and FastAPI behaviour.

16. Application scale and build verification

The catalogue and route counts are build contract checks rather than business success metrics. They confirm that repeated product pages are generated and that the application contains the intended navigation and service routes without inflating the dataset for appearance.

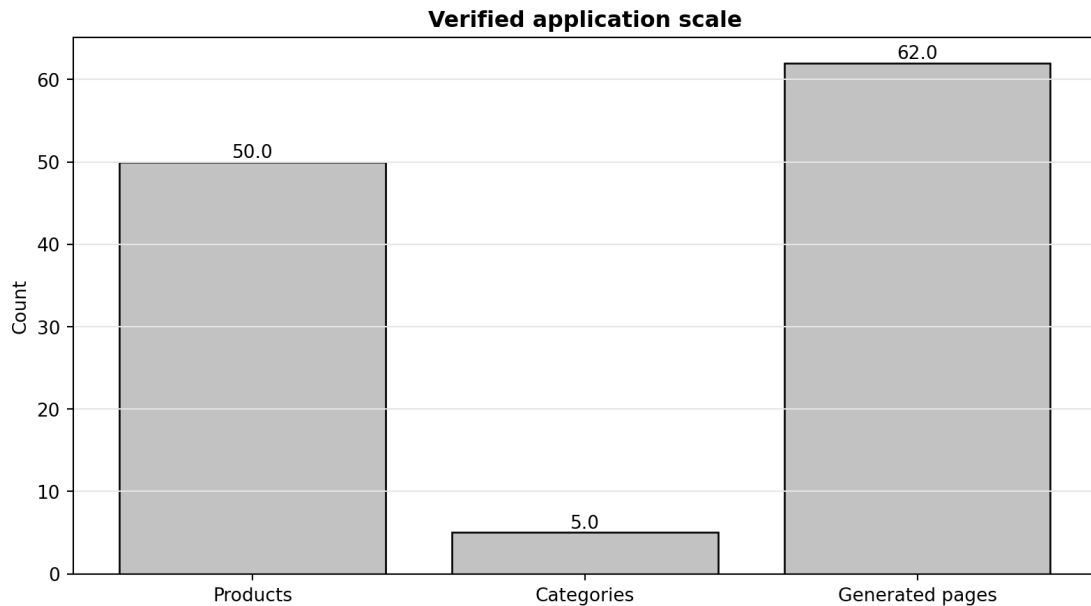


Figure 7. Product, category and generated page counts in the production build.

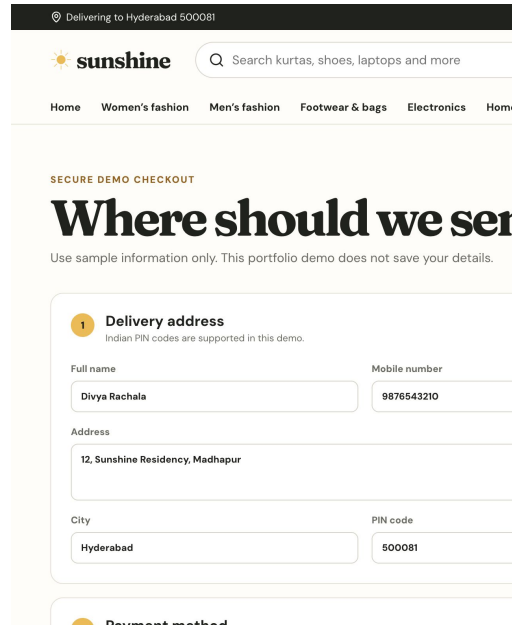
Test question: Does the build preserve the declared catalogue and generated page scope?

Observed result: The verified build contains 50 products, five categories and 62 generated or dynamic pages.

Interpretation: The scope exercises reusable product and route behaviour while keeping every generated page connected to the same catalogue rules.

17. Interface verification evidence

Browser evidence checks whether the implemented business rules remain understandable at the customer interface. The checkout capture shows the Indian delivery fields, three payment choices and explicit success and failure controls. Separate repository captures document the storefront, product recommendations, profile, recent orders and unavailable product state.



The screenshot displays the 'SECURE DEMO CHECKOUT' page of the Sunshine Retail Platform. At the top, a dark header shows the delivery location 'Delivering to Hyderabad 500081'. Below this is the Sunshine logo and a search bar. A navigation menu includes 'Home', 'Women's fashion', 'Men's fashion', 'Footwear & bags', 'Electronics', and 'Home'. The main heading is 'Where should we ser' (partially visible). A subheading reads 'Use sample information only. This portfolio demo does not save your details.' The form is titled '1 Delivery address' and includes a note: 'Indian PIN codes are supported in this demo.' The form fields are: 'Full name' (Divya Rachala), 'Mobile number' (9876543210), 'Address' (12, Sunshine Residency, Madhapur), 'City' (Hyderabad), and 'PIN code' (500081). A 'Payment method' section is partially visible at the bottom.

Figure 8. Checkout evidence with address, payment method and reproducible outcome controls.

Test question: Can each order branch be reproduced without entering real payment information?

Observed result: The form collects only temporary address data, labels the payment as simulated and provides successful, declined and unavailable outcomes.

Interpretation: The control is intentionally visible because it makes success and safe failure evidence repeatable in a public project.

18. Failure analysis

The most important negative test is not the message shown after failure; it is the absence of an unsafe side effect. For out of stock, payment must remain unattempted. For payment failure, the temporary stock hold must be released and delivery must remain uncreated. For an unavailable conversational model, product and order support must continue through bounded local rules.

The implementation records each failed attempt in order history. This choice makes the path reviewable and gives the assistant an exact status to report. It also avoids the misleading behavior of redirecting the customer back to the catalogue without explaining what happened.

Failure	Required safe behavior	Evidence
Quantity unavailable	Reject reservation and skip payment	Rejected snapshot, stopped timeline and failed trace
Payment declined	Release reservation and create no shipment	Released snapshot and stopped delivery stages
Ollama unavailable	Use bounded website support	Fallback intent tests and verified catalogue actions
Unknown order number	Do not invent a status	Exact order lookup response

19. Limitations and responsible interpretation

All products, ratings, payments, customers, inventory quantities, orders and courier events are synthetic. Browser storage is personal to one visitor and is not shared warehouse inventory. The public Vercel deployment may use hosted TypeScript adapters rather than the local Java and Python services.

The project does not contain production authentication, database transactions, payment settlement, fraud detection, reservation expiry, courier webhooks, returns processing or operational monitoring. The assistant is limited to website support and must not be interpreted as a general purpose shopping or financial adviser.

These boundaries identify exactly what was tested: coherent contracts, state transitions, orchestration and customer explanations.

The analytical measures also require careful interpretation. Twelve rows are sufficient to verify formulas and reporting structure, but they are not evidence of real market demand, city performance or commercial revenue. The values are reported only as outcomes of the declared synthetic fixture.

Security testing is limited to server only handling of the optional model key, validated order input and the absence of real payment credentials. Production review would also require dependency scanning, rate limiting, access control, audit logs, privacy assessment and incident response procedures.

20. Reproducibility and future work

The repository contains the Next.js application, Java and Python services, Docker files, API contracts, synthetic source data, KPI output, 23 automated tests, browser captures, seven generated experiment figures and this report. Each language suite can run independently, or the combined local stack can start with Docker Compose.

The next stage would add a shared relational database, authenticated accounts, idempotent order creation, expiring reservations, payment webhooks, a courier event simulator and end to end service tracing. Those features should be added only with new contract and failure tests, because additional components would otherwise reintroduce the disconnected state problem this project addresses.

Reproduction begins with the automated suites because they establish the business contract before a visual review. The production Next.js build then verifies the generated routes and TypeScript types. Java and Python can run separately, so the order and recommendation services do not require the optional conversational model.

Generated reports and figures are treated as outputs rather than hidden manual calculations. The KPI JSON is read directly by the report builder, and the figure scripts declare the test counts and scenario states they visualise. This keeps the written result consistent with the repository evidence.

21. Conclusion

Sunshine demonstrates a connected retail order rather than only an ecommerce interface. The strongest result is the agreement between the customer milestone, inventory snapshot, internal agent trace and automated tests for all three checkout branches.

The project also connects Java orchestration, Python recommendations and analytics, Next.js customer interaction and bounded conversational support without presenting synthetic behavior as production commerce. This combination keeps the implementation explainable and reproducible.

The implementation connects frontend state management, typed API design, Java responsibility boundaries, Python ranking and analysis, language model guardrails, failure testing and technical communication through one coherent problem. Every result is supported by a file, test, figure or customer flow.

Most importantly, the project treats failed payment and unavailable stock as first class outcomes. Correct recovery is part of the product, not an error message added after the successful path. That design decision makes the prototype a complete order lifecycle study.

For the confirmed branch, the evidence can be followed from the checkout request to the final screen. The requested quantity is available, a reservation is created, risk rules pass, payment is approved, the quantity is committed, picking becomes current and the remaining delivery stages are scheduled. The same order is saved for profile lookup and conversational support.

For the declined branch, the evidence follows a different path. The quantity is initially reserved, payment fails, the held units return to availability and neither fulfilment nor delivery is created. The cart remains available for another attempt. For the unavailable branch, reservation fails before risk or payment, which proves that the project avoids charging for a quantity it cannot supply.

The recommendation and analytics parts support the order lifecycle without replacing it. Recommendations resolve only to declared catalogue records, while the KPI workflow counts failed outcomes in the order funnel and excludes them from revenue. Both choices preserve the same principle: derived information must agree with the underlying commerce state.

My final assessment is that Sunshine meets its declared purpose. It does not solve production retail operations, but it does provide a complete and testable explanation of how a customer request becomes a confirmed, failed or rejected order. The combination of customer evidence and technical evidence is the main result of the work.